

```

# -*- coding: utf-8 -*-
"""
Created on Tues Sep 18 19:56:28 2018

@author: jfowl
"""
import numpy as np
import matplotlib.pyplot as plt
import control as ctl
import scipy.signal as sig

print('\n')

tvec = np.linspace(0, 20, 1000)

def analytic_sol20a(t):
    """x'(t) + 7x(t) = 5cos(2t)"""
    c1 = -35/53
    c2 = 35/53
    c3 = 10/53
    first = c1*np.exp(-7*t)
    second = c2*np.cos(2*t)
    third = c3*np.sin(2*t)
    x = first + second + third
    return x

an_20a = [tvec, [analytic_sol20a(t) for t in tvec]]
N20a, D20a = sig.zpk2tf([0], [-7, 2j, -2j], [5])
sys20a = ctl.tf(N20a, D20a)
tout20a, yout20a = ctl.impulse_response(sys20a, tvec)
print('Problem 20a)\n')
print('Transfer Function of {}: \n'.format(analytic_sol20a.__doc__), sys20a)
print('\n')
fig = plt.figure()
ax = fig.add_subplot(111)
ax.plot(an_20a[0], an_20a[1], 'r', linewidth=5,
        label='Analytic Solution')
ax.plot(tout20a, yout20a, '-.k', linewidth=2,
        label='Laplace Transform Solution')
plt.xlim([min(an_20a[0]), max(an_20a[0])])
plt.ylim([min(an_20a[1]), max(an_20a[1])])
handles, labels = ax.get_legend_handles_labels()
lgnd = ax.legend(handles, labels, loc='upper left',
                 bbox_to_anchor=(0.6, -0.05))
plt.xlabel('t-Value', fontsize=16)
plt.ylabel('x-Value', fontsize=16)
plt.title('Plot of the Solution to {}'.format(analytic_sol20a.__doc__),
          fontsize=18)
plt.grid(b=True, which='both')
fig.savefig('ME426_HW_4_20a', bbox_extra_artists=(lgnd,), bbox_inches='tight')
plt.show()

def analytic_sol20b(t):
    """x''(t) + 6x'(t) + 8x(t) = 5sin(3t)"""

```

```

c = 75/130
c1 = -39/130
c2 = -2/130
c3 = -36/130
first = c*np.exp(-2*t) + c1*np.exp(-4*t)
second = c2*np.sin(3*t)
third = c3*np.cos(3*t)
x = first + second + third
return x

an_20b = [tvec, [analytic_sol20b(t) for t in tvec]]
N20b, D20b = sig.zpk2tf([], [-4, -2, 3j, -3j], [15])
sys20b = ctl.tf(N20b, D20b)
tout20b, yout20b = ctl.impulse_response(sys20b, tvec)
print('Problem 20b\n')
print('Transfer Function of {}: \n'.format(analytic_sol20b.__doc__), sys20b)
print('\n')
fig = plt.figure()
ax = fig.add_subplot(111)
ax.plot(an_20b[0], an_20b[1], 'r', linewidth=5,
        label='Analytic Solution')
ax.plot(tout20b, yout20b, '-.k', linewidth=2,
        label='Laplace Transform Solution')
plt.xlim([min(an_20b[0]), max(an_20b[0])])
plt.ylim([min(an_20b[1]), max(an_20b[1])])
handles, labels = ax.get_legend_handles_labels()
lgnd = ax.legend(handles, labels, loc='upper left',
                 bbox_to_anchor=(0.6, -0.05))
plt.xlabel('t-Value', fontsize=16)
plt.ylabel('x-Value', fontsize=16)
plt.title('Plot of the Solution to {}'.format(analytic_sol20b.__doc__),
          fontsize=18)
plt.grid(b=True, which='both')
fig.savefig('ME426_HW_4_20b', bbox_extra_artists=(lgnd,), bbox_inches='tight')
plt.show()

def analytic_sol20c(t):
    """x'(t) + 8x(t) + 25x(t) = 10u(t)"""
    c = 10/25
    c1 = 8/15
    first = -np.exp(-4*t)
    second = c*np.cos(3*t)
    third = c1*np.sin(3*t)
    x = first*(second + third) + c
    return x

an_20c = [tvec, [analytic_sol20c(t) for t in tvec]]
N20c, D20c = sig.zpk2tf([], [0, -4-3j, -4+3j], [10])
sys20c = ctl.tf(N20c, D20c)
tout20c, yout20c = ctl.impulse_response(sys20c, tvec)
print('Problem 20c\n')
print('Transfer Function of {}: \n'.format(analytic_sol20c.__doc__), sys20c)
print('\n')
fig = plt.figure()

```

```

ax = fig.add_subplot(111)
ax.plot(an_20c[0], an_20c[1], 'r', linewidth=5,
        label='Analytic Solution')
ax.plot(tout20c, yout20c, '-.k', linewidth=2,
        label='Laplace Transform Solution')
plt.xlim([min(an_20c[0]), max(an_20c[0])])
plt.ylim([min(an_20c[1]), max(an_20c[1])])
handles, labels = ax.get_legend_handles_labels()
lgnd = ax.legend(handles, labels, loc='best',
                 bbox_to_anchor=(0.5, 0.5))
plt.xlabel('t-Value', fontsize=16)
plt.ylabel('x-Value', fontsize=16)
plt.title('Plot of the Solution to {}'.format(analytic_sol20c.__doc__),
          fontsize=18)
plt.grid(b=True, which='both')
fig.savefig('ME426_HW_4_20c', bbox_extra_artists=(lgnd,), bbox_inches='tight')
plt.show()

```

```

def analytic_sol21a(t):
    """x''(t) + 2x'(t) + 2x = sin(2t)"""
    c = 1/5
    c1 = 11
    c2 = -3
    c3 = -1/10
    c4 = 2
    first = c*np.exp(-t)
    second = c1*np.cos(t)
    third = c2*np.sin(t)
    fourth = c4*np.cos(2*t)
    fifth = np.sin(2*t)
    x = first*(second + third) + c3*(fourth + fifth)
    return x

```

```

an_21a = [tvec, [analytic_sol21a(t) for t in tvec]]
N21a, D21a = sig.zpk2tf([-0.7212, 0.1106-2.0365j, 0.1106+2.0365j],
                        [-1+1j, -1-1j, 2j, -2j], 2)
sys21a = ctl.tf(N21a, D21a)
tout21a, yout21a = ctl.impulse_response(sys21a, tvec)
print('Problem 21a)\n')
print('Transfer Function of {}: \n'.format(analytic_sol21a.__doc__), sys21a)
print('\n')
fig = plt.figure()
ax = fig.add_subplot(111)
ax.plot(an_21a[0], an_21a[1], 'r', linewidth=5,
        label='Analytic Solution')
ax.plot(tout21a, yout21a, '-.k', linewidth=2,
        label='Laplace Transform Solution')
plt.xlim([min(an_21a[0]), max(an_21a[0])])
plt.ylim([min(an_21a[1]), max(an_21a[1])])
handles, labels = ax.get_legend_handles_labels()
lgnd = ax.legend(handles, labels, loc='upper left',
                 bbox_to_anchor=(0.6, -0.05))
plt.xlabel('t-Value', fontsize=16)
plt.ylabel('x-Value', fontsize=16)
plt.title('Plot of the Solution to {}'.format(analytic_sol21a.__doc__),

```

```

        fontsize=18)
plt.grid(b=True, which='both')
fig.savefig('ME426_HW_4_21a', bbox_extra_artists=(lgnd,), bbox_inches='tight')
plt.show()

```

```

def analytic_sol21b(t):
    """x'(t) + 2x(t) + x = 5e^-2t + t"""
    first = -2
    second = t
    third = 5*np.exp(-2*t)
    fourth = 9*t*np.exp(-t)
    fifth = -np.exp(-t)
    x = first + second + third + fourth + fifth
    return x

```

```

an_21b = [tvec, [analytic_sol21b(t) for t in tvec]]
sys21b = ctl.TransferFunction([2, 9, 15, 1, 2], [1, 4, 5, 2, 0, 0])
tout21b, yout21b = ctl.impulse_response(sys21b, tvec)
print('Problem 21b)\n')
print('Transfer Function of {}:\n'.format(analytic_sol21b.__doc__), sys21b)
print('\n')
fig = plt.figure()
ax = fig.add_subplot(111)
ax.plot(an_21b[0], an_21b[1], 'r', linewidth=5,
        label='Analytic Solution')
ax.plot(tout21b, yout21b, '-.k', linewidth=2,
        label='Laplace Transform Solution')
plt.xlim([min(an_21b[0]), max(an_21b[0])])
plt.ylim([min(an_21b[1]), max(an_21b[1])])
handles, labels = ax.get_legend_handles_labels()
lgnd = ax.legend(handles, labels, loc='upper left',
                 bbox_to_anchor=(0, 1))
plt.xlabel('t-Value', fontsize=16)
plt.ylabel('x-Value', fontsize=16)
plt.title('Plot of the Solution to {}'.format(analytic_sol21b.__doc__),
          fontsize=18)
plt.grid(b=True, which='both')
fig.savefig('ME426_HW_4_21b', bbox_extra_artists=(lgnd,), bbox_inches='tight')
plt.show()

```

```

def analytic_sol21c(t):
    """x'(t) + 4x = t^2"""
    c = 9/8
    c1 = 1/4
    c2 = -1/8
    first = c*np.cos(2*t)
    second = np.sin(2*t)
    third = c1*t**2
    fourth = c2
    x = first + second + third + fourth
    return x

```

```

an_21c = [tvec, [analytic_sol21c(t) for t in tvec]]

```

```

sys21c = ctl.TransferFunction([1, 2, 0, 0, 2], [1, 0, 4, 0, 0, 0])
tout21c, yout21c = ctl.impulse_response(sys21c, tvec)
print('Problem 21c)\n')
print('Transfer Function of {}: \n'.format(analytic_sol21c.__doc__), sys21c)
print('\n')
fig = plt.figure()
ax = fig.add_subplot(111)
ax.plot(an_21c[0], an_21c[1], 'r', linewidth=5,
        label='Analytic Solution')
ax.plot(tout21c, yout21c, '-.k', linewidth=2,
        label='Laplace Transform Solution')
plt.xlim([min(an_21c[0]), max(an_21c[0])])
plt.ylim([min(an_21c[1]), max(an_21c[1])])
handles, labels = ax.get_legend_handles_labels()
lgnd = ax.legend(handles, labels, loc='upper left',
                 bbox_to_anchor=(0, 1))
plt.xlabel('t-Value', fontsize=16)
plt.ylabel('x-Value', fontsize=16)
plt.title('Plot of the Solution to {}'.format(analytic_sol21c.__doc__),
          fontsize=18)
plt.grid(b=True, which='both')
fig.savefig('ME426_HW_4_21c', bbox_extra_artists=(lgnd,), bbox_inches='tight')
plt.show()

```